



**UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA
FACULTAD DE INGENIERÍA MEXICALI**



Ingeniería en Computación

Trabajo:

Práctica 11: Comandos Avanzados de Vi.

Asignatura:

Taller de Sistemas Operativos Unix.

Docente:

Jesus Albany Armenta Garcia.

Alumno(a):

Limon Orozco Yael Alfonso

Matrícula:

01197973

Mexicali B.C. 14 de Octubre del 2025.

	UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA FACULTAD DE INGENIERÍA
	Formato de Práctica de Laboratorio

Programa Educativo		Plan de Estudios	Clave de Unidad de Aprendizaje	Nombre de la Unidad de Aprendizaje
Ingeniero en Computación		2020-1	36303	Taller de Sistema Operativo Unix
Práctica No.	Laboratorio de:	Taller de Sistema Operativo Unix		Duración (Horas)
11	Nombre de la Práctica:	Comandos Avanzados de Vi		2

1. INTRODUCCIÓN

Llegado a esta práctica, el alumno posee conocimientos básicos de cómo realizar las funciones básicas que otros editores de texto poseen. Sin embargo, vi ofrecen un conjunto de funciones que no se encuentran en editores convencionales, tales como la definición de macros, abreviaciones y configuraciones del documento a las que se puede acceder por medio de sencillos comandos los cuales serán repasados y utilizados en esta práctica.

2. OBJETIVO (COMPETENCIA)

El alumno conocerá y utilizará las funciones avanzadas que ofrece el editor de texto vi, aplicándolas de forma eficiente para la edición de textos con actitud positiva.

3. FUNDAMENTO

El Comando :set

En el editor vi es posible cambiar opciones que permiten la personalización del editor. Dichas opciones pueden ser lo que se conoce como *toggle options*, es decir, opciones de encendido o apagado, o bien, opciones en las que se tienen que definir valores numéricos o de texto.

```

:set option

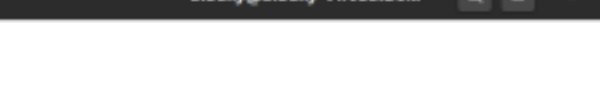
```

```
:set nooption
```

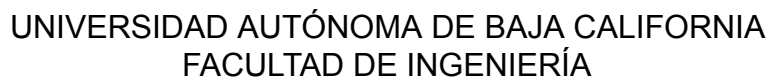
```
:set all
```

[illegible]

:set opción?



A screenshot of a terminal window. The title bar at the top reads "albany@albany-VirtualBox: ~". The terminal shows a series of blue vertical bars on the left side, indicating a command prompt. The prompt is followed by the text "set number".



Mapeo

	UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA FACULTAD DE INGENIERÍA Formato de Práctica de Laboratorio
---	--

En ocasiones, mientras se está utilizando el editor se puede dar el caso de estar utilizando una secuencia de comandos de manera frecuente, la cual puede ser una secuencia compleja o larga de escribir, lo que vendría a ser lo que se conoce como *macro* en algunos programas. Debido a esto, vi ofrece la capacidad de poder asignar una secuencia a una tecla no utilizada por medio del comando *map*:

:map x secuencia

definiendo la tecla x para comenzar la secuencia de comandos. La tecla que se defina para un macro no deberá estar asociada a alguna instrucción en modo comando. En caso de desear utilizar alguna tecla reservada, se deberá previamente presionar la tecla CTRL seguido de la tecla correspondiente a una letra que se quiere asociar a dicho macro. Por ejemplo, en caso de querer asociar un macro a la tecla k, la cual sube una línea estando en modo comando, y se desea que dicho macro escriba en el archivo la palabra *Universidad Autónoma de Baja California* justo al principio del archivo, la sintaxis sería la siguiente:

:map ^k ^[ggi^M^[ggiUniversidad Autonoma de Baja California^M^]

Cabe destacar que ^M y ^[no indican que se haya presionado la tecla CTRL seguido de la tecla M o [. En caso de al estar definiendo un macro se desea presionar la tecla ENTER (indicando una nueva línea en modo inserción) o la tecla ESC (para regresar a modo comando), se tiene que presionar CTRL+V (CTRL+q, en caso de estar en subsistema WSL) antes de presionar ENTER o ESC, ya que estas teclas tienen un comportamiento determinado en modo comando. Al presionar CTRL+V seguido de ENTER, en vi veremos reflejada dicha combinación como ^M, mientras que CTRL+V seguido de ESC se reflejará como ^[.

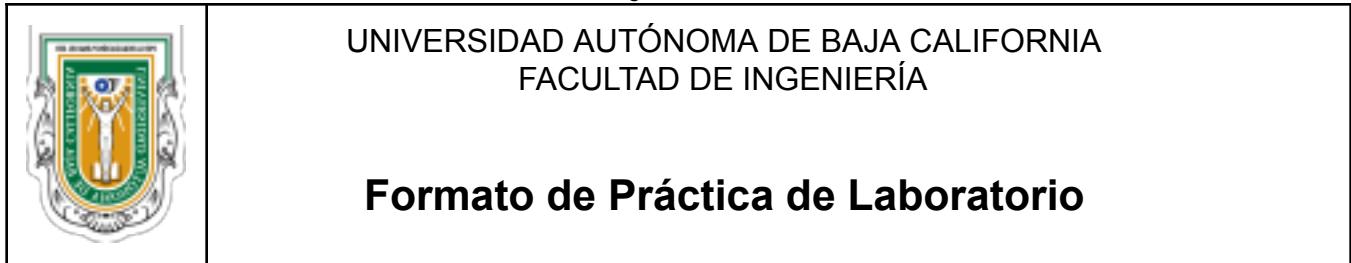
Para obtener un listado de los macros definidos, basta con escribir el comando

:map

Lo visto hasta este momento para mapeo únicamente funciona para cuando se está en modo comando, sin embargo, es posible definir macros también para modo inserción. Por ejemplo,

si agregáramos el caracter ! justo después de map de la siguiente forma: `:map! @`

`^[ggi^M^[ggiAlbany Armenta^M^]`



Se escribiría Albany Armenta justo al inicio del archivo en caso de presionar la tecla @ en modo inserción sin importar en que parte del archivo nos encontremos.

Para borrar un macro, basta con escribir en modo comando:

`:unmap ^k` para eliminar el macro que se ejecuta al presionar CTRL k en modo comando.

`:unmap! @` para eliminar el macro que se ejecuta al presionar @ en modo inserción. **EI**

Archivo .exrc

Las configuraciones, macros y abreviaciones definidas hasta el momento son temporales, es decir, al momento de salir del editor vi todas estas definiciones se pierden, por lo tanto, cada vez que utilizamos el editor habría que volver a definir las. En caso de desear guardarlas, todas estas deberán ser añadidas en un archivo llamado `.exrc` situado en el directorio *home* del usuario. De esta forma, la siguiente vez que se acceda a vi los comandos definidos en dicho archivo serán ejecutados automáticamente.

A continuación se muestra un ejemplo del contenido de un archivo `.exrc` con algunos comandos típicos:

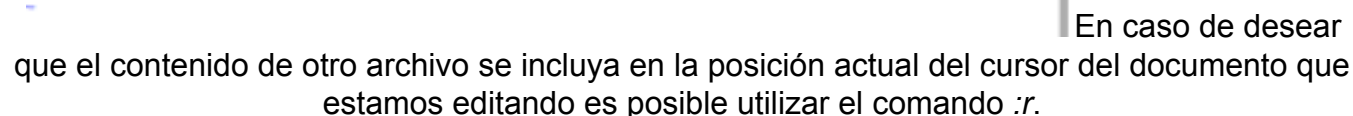
```
1 set nu
2 set sm
3 "se definen abreviaciones
4 ab nom Albany Armenta
5 ab inst Universidad Autonoma de Baja California
6 "se define un macro
7 map q ^[:wq^M
```

1. Establece la numeración automática de líneas en el editor.
2. Activa la opción *showmatch*, lo que permite que cuando se teclee un },) o] se muestre su respectivo caracter de apertura en caso de existir.
3. Comentario
4. Define la abreviación *nom* que escribirá el nombre Albany Armenta.
5. Define la abreviación para poner el nombre de la universidad en caso de escribir *inst*.

- Se crea un macro para que cuando se presione la tecla q desde modo comando, se pase a modo comando en caso de no estarlo (redundante) y guarde los cambios en el archivo que se está editando y cerrando vi en el proceso.

Desde vi es posible utilizar comandos de shell de Linux. Para ello, solo se tiene que escribir lo siguiente en modo edición de línea:

En donde el símbolo `!` le indica al editor que lo que sigue es un comando propio del shell de Linux y es ejecutado una vez se presiona la tecla Enter. De ser necesario integrar al documento el resultado de la ejecución de un comando, se utiliza el comando de vi `:r!` tal como se observa en la siguiente imagen, en donde se agrega al documento el resultado de ejecutar el comando `date` en la línea siguiente de donde se encuentra el cursor.



4. PROCEDIMIENTO

Equipo Necesario	Material de Apoyo
------------------	-------------------

- Computadora con sistema operativo Ubuntu instalado.

- Formato de Práctica

Página 6 de 8

	UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA FACULTAD DE INGENIERÍA Formato de Práctica de Laboratorio
---	--

Desarrollo de la Práctica

1. Inicie sesión ssh en el servidor de computación.
2. Cada macro y abreviación que se realice a partir de este paso deberá ser registrado en un archivo `.exrc`.
3. Define abreviaciones para representar cada uno de tus datos personales que se enlistan a continuación, anotado cada una de las instrucciones utilizadas para definir cada abreviación.
 - a) Nombre Completo:
dentro del modo comando introduce **:ab yael Yael Alfonso Limon Orozco**
 - b) Programa Educativo:
dentro del modo comando introduce **:ab compu Ingenieria en Computacion**
 - c) Matrícula:
dentro del modo comando introduce **:ab mat 01197973**
4. Redefine la tecla `g` para que al ser presionada en modo comando, permita borrar o eliminar la palabra completa donde se encuentre el cursor actualmente.

:map g dw

5. Mapee la tecla F3 para que al ser presionada en modo comando, intercambie dos líneas del documento, la línea donde se encuentra el cursor pasará a estar un renglón arriba y la de arriba quedará un renglón abajo.

:map <F3> ddkp

6. Cree un macro para que al presionar la tecla ~ en modo inserción se escriba en el documento su dirección de correo electrónico justo donde se encuentra el cursor.

:map! ~ yael.limon@uabc.edu.mx

7. Realice un macro que al presionar la tecla F4 borre la última palabra de la línea y la pegue al inicio de la misma, justo antes de la primera palabra, colocando un espacio entre ambas palabras.

:map <F4> \$bdw0Pli ^[(tecla ESC)

8. Muestra una lista con todas las teclas definidas para ser utilizadas en modo comando.

:map

```
x  gx          <Plug>NetrwBrowseXVis
n  gx          <Plug>NetrwBrowseX
   g           dw
x  <Plug>NetrwBrowseXVis * :<C-U>call netrw#BrowseXVis()<CR>
n  <Plug>NetrwBrowseX * :call netrw#BrowseX(netrw#GX(),netrw#CheckIfRemote(n
etrw#GX()))<CR>
   <F4>        $bdw0Pli <Esc>
   <F3>        ddkp
Pulse INTRO o escriba una orden para continuar
```

9. Muestra una lista con todas las teclas definidas para ser utilizadas en modo inserción.

:map!

```
!  ~           yael.limon@uabc.edu.mx           3,1           Todo
```

10. Cree un archivo a partir del comando *finger* que contenga únicamente el usuario de las personas conectadas al servidor. Este archivo deberá nombrarse *nombres*.

```
limon.yael@computacion:~$ finger | cut -d' ' -f1 | tail -n -1 > nombres
limon.yael@computacion:~$ cat nombres
limon.yael
```

11. La instrucción `:%g/^$/d` en modo comando elimina todas las líneas en blanco del documento que se está editando en vi. Cree un macro con la tecla F5 para que elimine todas las líneas en blanco, es decir, sin información, del archivo *nombres* creado en el paso anterior.

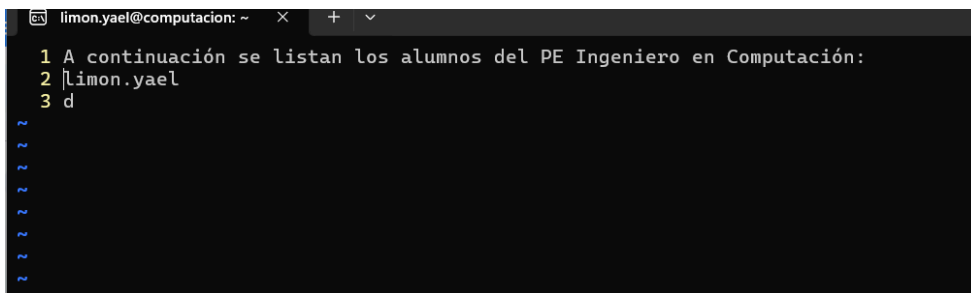
`map <F5> :%g/^$/d^M` (tecla ENTER)

12. Cree un nuevo archivo en vi llamado *alumnosComputacion* y escriba “A continuación se listan los alumnos del PE Ingeniero en Computación” y sin salir de vi, inserte el contenido del archivo *nombres* un renglón abajo.

`:r nombres`

13. Redefine la tecla F7 para que al ser presionada muestre el número de línea para cada una de las líneas del archivo *alumnosComputacion*.

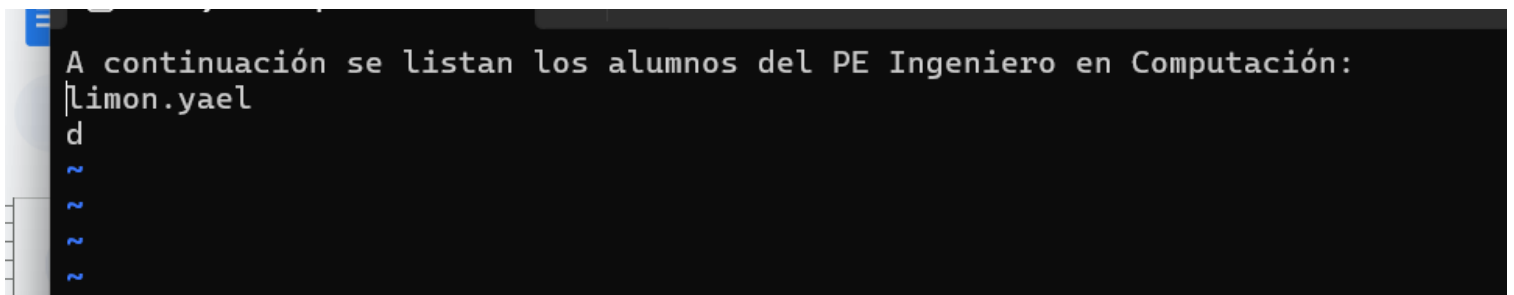
`:map <F7> :set number^M`



```
limon.yael@computacion: ~  
1 A continuación se listan los alumnos del PE Ingeniero en Computación:  
2 limon.yael  
3 d  
~  
~  
~  
~  
~  
~  
~  
~
```

14. Ahora redefine la tecla F8 para que al ser presionada oculte el número de líneas del archivo.

`:map <F8> :set nonumber^M`



```
A continuación se listan los alumnos del PE Ingeniero en Computación:  
limon.yael  
d  
~  
~  
~  
~  
~
```

15. Al final del archivo *alumnosComputacion* inserte la fecha del sistema con el respectivo comando de Linux.

`:r! date`

```
A continuación se listan los alumnos del PE Ingeniero en Computación:  
limon.Yael
```

```
|mar 21 oct 2025 12:59:20 PDT
```

```
~  
~
```

16. Termine la sesión.

5. RESULTADOS Y CONCLUSIONES

Dentro del editor vi se puede personalizar dependiendo del uso que le queramos dar, de tal forma que podremos definir a nuestro criterio como se observa y trabaja con el editor. Funciones que se trabajaron durante el desarrollo de esta practica las cuales considero más importantes fueron las de set y map, donde set nos ayuda a personalizar el editor y map que nos ayuda a crear una macro para una función en específica con solo una tecla, como tomar una secuencia de teclas y mapearla a una tecla disponible.

6. ANEXOS

7. REFERENCIAS

- A. Robbins, E. Hannah and A. Robbins, Learning the vi and Vim editors, 8th ed. 2021. Página 8 de 8